
Real-Time Object Detection and Counting from Video Streams

Full Report (Part 1)

P. O. Adepoju

AI/ML Engineering Portfolio Project

Model Family: YOLOv8 (Nano / Small / Medium)
Dataset: COCO 80-class pretrained weights
Task: Multi-class object detection & counting
Date: June 19, 2026

Abstract

Real-time video object detection is a foundational capability in modern artificial intelligence engineering, with applications ranging from urban traffic monitoring and warehouse automation to retail analytics and public safety surveillance. This report presents the design, implementation, and evaluation of a complete, end-to-end video object detection and counting system built using YOLOv8 (Jocher, Chaurasia, & Qiu, 2023) pretrained on the COCO 80-class dataset (Lin et al., 2014).

The system processes video streams frame-by-frame, producing annotated output videos with bounding boxes and class labels, exporting per-frame detection records as structured JSON, computing object count time-series across the full video duration, and benchmarking three model variants cross four engineering dimensions: inference latency, sustained throughput in frames per second (FPS), on-disk model size, and peak memory usage.

Applied to a 60-second 1920×1080 public-domain traffic video (1,501 frames at 25 FPS), the pipeline produced 10,725 total detections. Cars dominated with 7,776 cumulative appearances, followed by trucks (2,044, 82.7% of frames), persons (745, 31.2%), motorcycles (155, 10.0%), and a single bus (5, 0.3%).

Benchmarking under controlled CPU conditions revealed a clear monotonic relationship between model scale and inference cost: **YOLOv8-nano** achieved 9.58 FPS at a mean latency of 104.3 ms (p95 = 124.3 ms); **YOLOv8-small** achieved 4.37 FPS at 229.1 ms (p95 = 276.0 ms); and **YOLOv8-medium** achieved only 1.20 FPS at 831.7 ms with a heavily right-skewed distribution (p99 = 4,119 ms), rendering it unsuitable for real-time CPU deployment.

The primary contributions of this project is a fully reproducible pipeline covering environment setup, dataset exploration, single-frame detection, full-video inference, object counting, model benchmarking, model comparison, and an interactive Streamlit dashboard.

Limitations include CPU-only benchmarking, frame-level counting without temporal object tracking, zero-shot inference on COCO classes without domain fine-tuning, and the absence of formal mAP evaluation. These limitations are discussed in detail alongside a structured set of future extensions.

Keywords: YOLOv8, COCO, real-time object detection, bounding box regression, video inference pipeline, benchmarking, object counting, Ultralytics, computer vision, machine learning engineering.

Contents

Abstract	1
1 Introduction	3
1.1 Motivation	3
1.2 Project Goals and Scope	4
2 Background	5
2.1 A Brief History of Object Detection	5
2.1.1 Era 1: Handcrafted Features (Pre-2012)	5
2.1.2 Era 2: Deep Learning Two-Stage Detectors (2013–2016)	5
2.1.3 Era 3: Single-Stage Detectors (2016–Present)	6
2.1.4 Era 4: The YOLOv8 Lineage and Architecture	6
2.2 Mathematical Formulation	7
2.2.1 Task Definition	7
2.2.2 Intersection over Union (IoU)	7
2.2.3 Confidence Thresholding	8
2.3 The COCO Dataset and Class Taxonomy	8
3 Implementation	10
3.1 Single-Frame Detection	10
3.2 Full Video Inference Pipeline	12
3.3 Object Counting Logic	13
3.4 Model Benchmarking	14
3.5 Model Size Comparison and Deployment Recommendations	15
References	17

Chapter 1

Introduction

1.1 Motivation

According to IHS Markit, more than one billion surveillance cameras were in operation worldwide by 2021, the vast majority recording footage that is never meaningfully analysed (IHS Markit, 2022). The gap between *recording* and *understanding* is precisely the domain of computer vision engineering: the ability to extract structured, actionable information from raw video streams in real time.

Object detection has undergone a revolution since the introduction of deep convolutional neural networks (CNNs). Systems that once required minutes of computation per frame now run at tens or even hundreds of frames per second on commodity hardware, enabling a broad and growing set of deployment scenarios:

- **Traffic monitoring.** Counting vehicles per lane, measuring junction throughput, detecting violations (red-light running, wrong-way driving), and estimating queue lengths for adaptive signal control.¹
- **Warehouse automation.** Tracking inventory items on conveyor belts, detecting out-of-place objects, monitoring worker PPE (Personal Protective Equipment) compliance,² and verifying pallet counts.
- **Retail analytics.** Measuring customer dwell time in front of product displays, estimating shelf occupancy, detecting empty shelves for restocking alerts, and counting checkout queue lengths.
- **Safety and crowd monitoring.** Density estimation for egress planning, fall detection for elderly care, restricted zone enforcement in hazardous environments, and occupancy management for post-pandemic social distancing.³
- **Event analysis.** Occupancy measurement at concerts and stadiums, access control verification, and post-event crowd flow reconstruction.

Each application places different constraints on the detection system. A live traffic enforcement camera must process at or near the native video frame rate (typically 25–30 FPS) to avoid missing brief events. A warehouse inventory reconciliation system may be tolerant of a one-second lag. A post-event crowd analysis pipeline can process frames in batch mode overnight.

¹See the UK Department for Transport’s Automatic Traffic Count programme for a real-world deployment example: <https://www.gov.uk/government/statistical-data-sets/tra25-traffic-counts>.

²OSHA PPE compliance using computer vision is an active commercial sector; see, e.g., <https://viso.ai/computer-vision/ppe-detection/>.

³The British Standards Institution published PAS 185:2017 on smart city surveillance frameworks; applicable guidance at <https://www.bsigroup.com/>.

Pedagogical Note: What Does “Real-Time” Mean?

The term “real-time” is used loosely in the computer vision literature. A precise taxonomy has three levels (Buttazzo, 2011):

- **Hard real-time:** Missing a processing deadline causes system failure (e.g., flight control, nuclear plant monitoring). Object detection almost never falls in this category.
- **Soft real-time:** Missing a deadline degrades quality but does not cause failure (e.g., live video analytics, traffic counting). Most camera-based ML systems are here.
- **Best-effort:** No timing guarantee; the system processes as fast as it can (e.g., batch video analytics). Appropriate for post-event reconstruction.

The systems built in this project are best characterised as *soft real-time* targets. A mean latency of 104 ms per frame (YOLOv8-nano, CPU) means the system processes roughly 9.6 frames per second, below the 25 FPS native rate, but acceptable for many monitoring applications. The medium model at 1.2 FPS accumulates a 20× real-time lag and is therefore unsuitable for any live monitoring use case on CPU.

1.2 Project Goals and Scope

This project has two explicit, complementary goals:

1. **Engineering Goal.** Build a complete, production-quality, end-to-end video object detection and counting pipeline. The pipeline must accept a raw video file, run YOLOv8 inference on every frame, write an annotated output video, export per-frame detection records as JSON, compute per-class object count time-series, and serve results through an interactive Streamlit dashboard.
2. **Scientific Goal.** Systematically benchmark three YOLOv8 model variants (nano, small, medium) under controlled, reproducible conditions to quantify and characterise the speed–accuracy–memory trade-off.

In Scope. YOLOv8 nano, small, and medium variants from Ultralytics;⁴ COCO 80-class pretrained weights used zero-shot (no fine-tuning); CPU-only benchmarking; frame-level object counting.

Out of Scope. GPU benchmarking; domain-specific fine-tuning; temporal object tracking (SORT/ByteTrack);⁵ formal mAP evaluation on COCO val2017; large and xlarge model variants.

⁴Ultralytics YOLOv8 documentation: <https://docs.ultralytics.com/models/yolov8/>.

⁵SORT tracker: <https://github.com/abewley/sort>. ByteTrack: <https://github.com/ifzhang/ByteTrack>.

Chapter 2

Background

2.1 A Brief History of Object Detection

Object detection has evolved through three distinct eras over the past two decades, each characterised by the dominant feature representation paradigm.

2.1.1 Era 1: Handcrafted Features (Pre-2012)

Early object detectors relied on features designed by human experts. The seminal work of [Viola and Jones \(2001\)](#) introduced the *Viola-Jones face detector*: a cascade of simple rectangle-feature classifiers trained with AdaBoost.¹ The integral image data structure reduced feature computation to $O(1)$ per rectangle, making real-time face detection feasible on 2001-era hardware.

[Dalal and Triggs \(2005\)](#) introduced Histograms of Oriented Gradients (HOG) combined with a linear SVM for pedestrian detection. HOG descriptors encode local edge orientation statistics in a dense grid of spatial cells, capturing the overall shape of objects without modelling individual pixels. HOG + SVM dominated pedestrian detection benchmarks from 2005 through approximately 2012.

The fundamental limitation of both approaches was brittleness: features handcrafted for one object type (faces, pedestrians) generalised poorly to others. Building a general-purpose multi-class detector required a separate feature engineering pipeline for each class.

2.1.2 Era 2: Deep Learning Two-Stage Detectors (2013–2016)

The publication of AlexNet ([Krizhevsky, Sutskever, & Hinton, 2012](#)) at ILSVRC 2012 demonstrated that deep CNNs trained on large labelled datasets could learn feature hierarchies far more powerful than any handcrafted alternative. This catalysed a rapid transformation of detection.

[Girshick, Donahue, Darrell, and Malik \(2014\)](#) introduced **R-CNN** (Region-based CNN): a three-stage pipeline consisting of (1) selective search to generate $\sim 2,000$ region proposals per image, (2) a CNN to extract a 4,096-dimensional feature vector per proposal, and (3) class-specific SVMs for classification. R-CNN achieved 53.3 mAP on VOC 2010, a 30% relative improvement over the prior state of the art. Its limitation was speed: 47 seconds per image on a GPU, because the CNN was applied independently to each proposal with no feature sharing.

[Girshick \(2015\)](#) resolved the efficiency problem with **Fast R-CNN**: a shared convolutional backbone

¹A clear visual explanation of Haar features and the integral image trick is available at https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html.

extracts a single feature map for the entire image; proposals are then mapped onto this feature map via *RoI pooling* (Region of Interest pooling), which extracts a fixed-size feature vector from any rectangular region. This reduced inference time to ~ 2 seconds per image.

Ren, He, Girshick, and Sun (2015) completed the two-stage paradigm with **Faster R-CNN**, which replaced selective search with a *Region Proposal Network* (RPN) sharing the backbone’s features. The RPN predicts objectness scores and bounding box offsets at each spatial location of the feature map, across multiple pre-defined scales and aspect ratios called *anchor boxes*.² Faster R-CNN achieved ~ 5 FPS, a $9\times$ improvement over R-CNN.

2.1.3 Era 3: Single-Stage Detectors (2016–Present)

Two-stage detectors are accurate but slow because of the sequential proposal-then-classify pipeline. Liu et al. (2016) (SSD) and Redmon, Divvala, Girshick, and Farhadi (2016) (YOLOv1) simultaneously proposed eliminating the region proposal step entirely, predicting bounding boxes and class probabilities in a single forward pass through the network.

SSD predicts at multiple feature map resolutions: early (high-resolution, small receptive field) layers detect small objects; later (low-resolution, large receptive field) layers detect large objects. Predictions are made at each spatial location across all scales with a set of default boxes.

YOLOv1 divided the image into a grid of $S \times S$ cells. Each cell predicted B bounding boxes and C class probability distributions simultaneously. This achieved 45 FPS at 63.4 mAP on PASCAL VOC 2007, the first single-stage detector to approach two-stage accuracy. The key insight was framing detection as a single regression problem rather than a classification pipeline.

Redmon and Farhadi (2017) (YOLOv2) added batch normalisation, high-resolution classifiers, anchor boxes (borrowed from Faster R-CNN), and multi-scale training, achieving 76.8 mAP at 40 FPS. Redmon and Farhadi (2018) (YOLOv3) introduced Darknet-53 as the backbone and multi-scale prediction with three detection heads operating at three different feature map resolutions, directly enabling detection of objects at vastly different scales within the same image.

2.1.4 Era 4: The YOLOv8 Lineage and Architecture

After YOLOv3, the YOLO series fragmented across multiple research groups.³ YOLOv4 (Bochkovskiy, Wang, & Liao, 2020) introduced CSP (Cross-Stage Partial) bottlenecks, PANet (Path Aggregation Network) feature fusion, and a “Bag of Freebies” training protocol (mosaic augmentation, CIoU loss). YOLOv5 (Ultralytics) was a complete rewrite in PyTorch with AutoML hyperparameter optimisation. YOLOv6, v7, and v9 introduced various architectural improvements from concurrent groups.

Jocher et al. (2023) released **YOLOv8** in January 2023 as a ground-up redesign by Ultralytics. Key design

²A detailed tutorial on anchor boxes and the RPN is available from Towards Data Science: <https://towardsdatascience.com/anchor-boxes-the-key-to-quality-object-detection-ddf9d612d4f9>.

³A comprehensive comparison of YOLO versions (v1–v8) is available at Papers With Code: <https://paperswithcode.com/methods/category/object-detection>.

choices that distinguish YOLOv8 from its predecessors include:

- **Anchor-free decoupled head.** Separate branches for classification and box regression, each with their own convolutional layers. This contrasts with coupled heads in earlier versions, which was found to cause conflicting gradients between the two tasks.
- **C2f bottleneck.** A modified CSP bottleneck module with two bottleneck chains (“Cross Stage Partial with 2 feature fusion paths”), providing richer feature gradients.
- **DFL (Distribution Focal Loss).** Box regression is reformulated as a discrete distribution over box edge coordinates rather than a direct regression, yielding more calibrated predictions near the distribution boundary.
- **Python-first API.** A clean `model.predict()` interface returning structured `Results` objects, making integration into downstream Python applications straightforward.

This project uses three of the five available scale variants: nano ($\approx 3.2\text{M}$ parameters), small ($\approx 11.2\text{M}$ parameters), and medium ($\approx 25.9\text{M}$ parameters).

2.2 Mathematical Formulation

2.2.1 Task Definition

Definition 2.1 (Object Detection Task). Given an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, produce a set

$$\mathcal{D} = \{(b_i, c_i, s_i)\}_{i=1}^N$$

where $N \geq 0$ is the number of detected objects, and for each detection:

$$\begin{aligned} b_i &= (x_1^{(i)}, y_1^{(i)}, x_2^{(i)}, y_2^{(i)}) \in \mathbb{R}^4 \quad (\text{bounding box in pixel coordinates, top-left to bottom-right}), \\ c_i &\in \{0, 1, \dots, C-1\} \quad (\text{predicted class index, where } C = 80 \text{ for COCO}), \\ s_i &\in [0, 1] \quad (\text{confidence score: model certainty that class } c_i \text{ is present in } b_i). \end{aligned}$$

2.2.2 Intersection over Union (IoU)

Definition 2.2 (Intersection over Union). Given two bounding boxes A and B , their IoU is:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

where $|A|$ denotes the area of box A .

IoU is the canonical measure of spatial overlap between a predicted box and a ground-truth box. Under the COCO evaluation protocol (Lin et al., 2014), a detection is considered a *true positive* if it satisfies two conditions: (1) it predicts the correct class, and (2) its IoU with the matched ground-truth box exceeds a

threshold (commonly $\tau = 0.5$).

Example 2.1 (IoU Computation). Let $A = (100, 150, 300, 400)$ and $B = (200, 250, 450, 500)$ in xyxy format.

$$\begin{aligned}
 x_{1,\cap} &= \max(100, 200) = 200, & x_{2,\cap} &= \min(300, 450) = 300 \\
 y_{1,\cap} &= \max(150, 250) = 250, & y_{2,\cap} &= \min(400, 500) = 400 \\
 |A \cap B| &= (300 - 200)(400 - 250) = 100 \times 150 = 15,000 \\
 |A| &= 200 \times 250 = 50,000; & |B| &= 250 \times 250 = 62,500 \\
 \text{IoU}(A, B) &= \frac{15,000}{50,000 + 62,500 - 15,000} = \frac{15,000}{97,500} \approx 0.154
 \end{aligned}$$

Since $0.154 < 0.5$, detection B would not be counted as a true positive against ground truth A .

2.2.3 Confidence Thresholding

In this project $\tau_{\text{conf}} = 0.25$. The threshold controls the precision–recall trade-off:

- **Low threshold** (e.g., $\tau = 0.05$): higher recall (fewer missed objects) but lower precision (more false positives). At $\tau = 0.05$, the middle frame produces 35 detections (vs. 18 at $\tau = 0.25$).
- **High threshold** (e.g., $\tau = 0.70$): higher precision but lower recall. At $\tau = 0.70$, the same frame yields only 12 detections.

2.3 The COCO Dataset and Class Taxonomy

The COCO (Common Objects in Context) dataset (Lin et al., 2014) is the canonical benchmark for object detection.⁴ Key statistics: 330,000 images, 1.5 million annotated object instances across 80 categories, approximately 7 objects per image on average, with annotations consisting of bounding boxes, segmentation masks, and keypoints (for the human pose task).

The 80 COCO categories can be grouped into 12 semantic domains:

⁴The COCO dataset and its evaluation server are available at <https://cocodataset.org/>.

Table 2.1: COCO 80-class semantic taxonomy. Figure 2.1 shows the category size distribution.

Category Group	Count	Examples
People	1	person
Vehicles	8	bicycle, car, motorcycle, bus, train, truck
Traffic	4	traffic light, fire hydrant, stop sign, parking meter
Animals	10	bird, cat, dog, horse, elephant, bear
Sports	10	frisbee, skis, skateboard, surfboard, tennis racket
Food	10	banana, pizza, donut, hot dog, sandwich
Electronics	11	tv, laptop, cell phone, keyboard, microwave
Kitchen	7	bottle, wine glass, cup, fork, knife
Furniture	6	chair, couch, bed, dining table, toilet
Accessories	5	backpack, umbrella, handbag, tie, suitcase
Household	7	book, clock, vase, scissors, teddy bear
Outdoor	1	bench

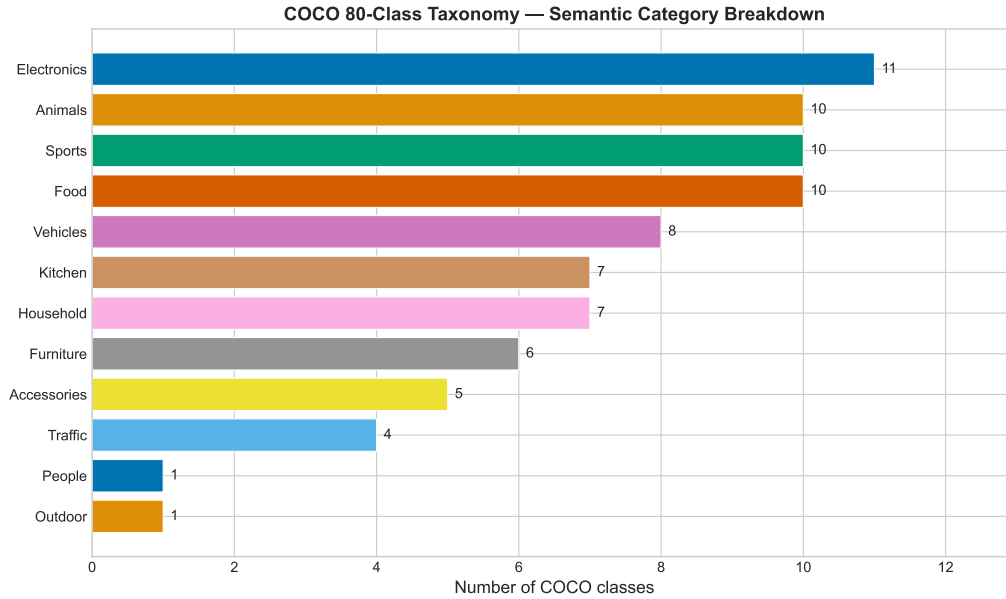


Figure 2.1: COCO 80-class semantic category breakdown, visualised as a horizontal bar chart. Categories are sorted by class count (descending). Vehicles, animals, sports, food, and electronics each contain 8–11 classes; people and outdoor each contain only one. Generated by *Notebook 02*.

For the traffic video used in this project, five classes appear in the detections: **car** (class index 2), **truck** (7), **person** (0), **motorcycle** (3), and **bus** (5). This is consistent with an urban road scene: the COCO “car” class covers passenger vehicles, “truck” covers large commercial vehicles and vans, and “motorcycle” covers two-wheeled motorised vehicles.

Chapter 3

Implementation

3.1 Single-Frame Detection

Notebook 03 runs detection on individual frames before committing to a full-video run. This is a standard ML engineering practice: inspect the output of a single forward pass exhaustively before scaling to thousands of frames where debugging becomes exponentially harder.

Frame Extraction. Three frames are extracted at 10%, 50%, and 90% of the video duration using `cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)`. Sampling at the beginning, middle, and end provides a representative cross-section of traffic conditions.

Single-Frame Detection. YOLOv8-small is loaded and run on the middle frame (50% position). The raw output inspected in the notebook is:

Table 3.1: Raw detection output on the middle frame (frame index 750, 50% position), YOLOv8-small, confidence threshold 0.25. Single-call inference time: 407.4 ms.

Det.#	Class	Confidence	Box (xyxy, px)
0	car	0.91	(142, 302, 521, 518)
1	car	0.90	(821, 287, 1124, 430)
2	car	0.81	(1401, 256, 1652, 381)
3	car	0.78	(528, 271, 741, 385)
4	car	0.78	(1061, 248, 1262, 366)
5	truck	0.48	(304, 221, 683, 358)
6	truck	0.38	(671, 228, 924, 347)
7	car	0.37	(671, 228, 924, 347)

Detections 6 and 7 have identical bounding boxes but different class labels (truck, car) and similar confidences (0.38, 0.37). This is a classic near-threshold ambiguity: the model assigns substantial probability mass to both classes for a large vehicle at moderate distance. Without NMS suppression between classes (NMS operates independently per class in YOLO), both predictions survive the threshold.



Figure 3.1: Left: the original middle frame (1920×1080, displayed at reduced resolution). Right: the same frame with YOLOv8-small detections overlaid. Eight detections are visible: six cars (blue boxes) and two trucks (orange boxes). The near-duplicate detection (detections 6 and 7 in Table 3.1) is visible as a single double-labelled box on the large vehicle in the centre-left. Generated by *Notebook 03*.

Confidence Distribution. Running detection with a low threshold ($\tau = 0.05$) on all three sampled frames produces a richer confidence distribution. Figure 3.2 shows histograms and box plots. The confidence summary is:

Table 3.2: Confidence score distribution by class across three sampled frames (threshold 0.05, YOLOv8-small).

Class	N	Mean	Std	Min	Median	Max
car	20	0.610	0.325	0.072	0.780	0.914
truck	9	0.238	0.188	0.052	0.218	0.563
person	6	0.066	0.010	0.053	0.069	0.075

Three observations merit attention:

- **Car confidences span the full range** (0.072 to 0.914). Clearly visible nearby vehicles produce high-confidence detections; partially occluded or distant vehicles produce low-confidence detections near the threshold.
- **Truck confidences are systematically lower** (mean 0.238). The COCO “truck” class was trained primarily on large articulated lorries; the medium-sized commercial vans visible in the traffic video partially match but do not fully activate the truck detector, producing moderate confidence scores.
- **Person confidences cluster near the threshold** (0.053–0.075). At 1920×1080 and the camera position used, pedestrians occupy small image regions (<1% of the frame area). The model detects them but with low confidence, consistent with the known difficulty of detecting small objects far from the camera.¹

¹Small object detection is an active research area. YOLOv8’s stride-8 detection head is specifically designed for small objects; see <https://arxiv.org/abs/2304.00501> for the full YOLOv8 technical report.

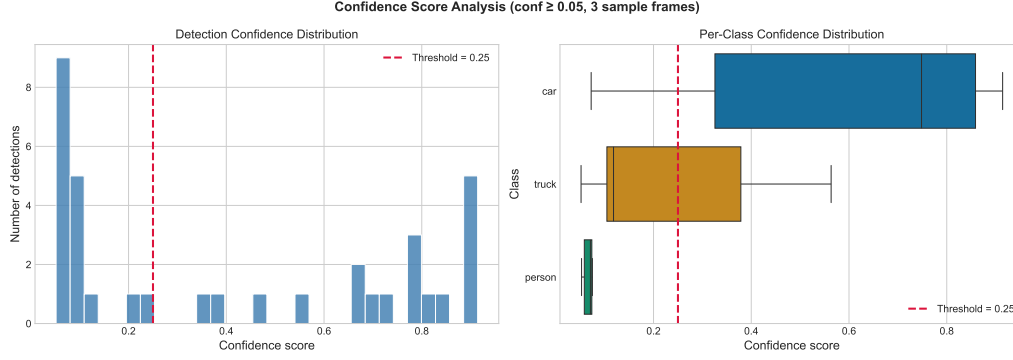


Figure 3.2: Confidence score distributions for detections across three sampled frames (confidence threshold 0.05). Left panel: histogram of confidence scores coloured by class. Right panel: per-class box plot showing median, IQR, and outliers. Car detections are broadly distributed; truck and person detections cluster near the threshold. Generated by *Notebook 03*.

3.2 Full Video Inference Pipeline

Notebook 04 processes all 1,501 frames of the traffic video end-to-end.

Verbose Log Parsing. With `verbose=True` (enabled in *Notebook 04* for the first few frames), Ultralytics prints a structured line per frame:

```
0: 384x640 7 cars, 2 trucks, 238.4ms
Speed: 5.9ms preprocess, 238.4ms inference, 2.4ms postprocess per image at shape
      (1, 3, 384, 640)
```

The three speed components deserve attention:

- **Preprocess (5.9 ms):** letterbox resizing, normalisation to $[0, 1]$, and conversion to a PyTorch tensor. This is dominated by OpenCV resize operations.
- **Inference (238.4 ms):** the neural network forward pass through the full CSP backbone, PANet neck, and three detection heads. This is by far the dominant cost.
- **Postprocess (2.4 ms):** applying the confidence threshold and NMS, and converting tensor outputs to the `Results` object structure.

Latency Timeline. Figure 3.3 shows the per-frame FPS and latency over all 1,501 frames.



Figure 3.3: Top panel: per-frame FPS (thin line) with 10-frame rolling mean (thick line) over the full 1,501-frame run (YOLOv8-small, CPU). Bottom panel: per-frame latency in ms. Mean FPS: 4.8; mean latency: 222.0 ms. Significant latency spikes (one reaching 4,108.6 ms) are visible, attributed to OS scheduling interrupts and thermal throttling. Generated by *Notebook 04*.

The latency statistics for the full video run are:

Statistic	Value
Mean latency	222.0 ms
Median latency	201.2 ms
Std deviation	120.8 ms
p95 latency	313.7 ms
p99 latency	464.7 ms
Mean FPS	4.8
Max latency	4,108.6 ms

3.3 Object Counting Logic

Notebook 05 loads the `detections.json` file produced by *Notebook 04* and applies the counting pipeline to produce per-class count time-series and summary statistics.

Count Time-Series. Figure 3.4 shows the per-frame count for the top 5 detected classes over the full 60-second video. The car count (blue line, top panel) is the most prominent, oscillating between 2 and 9 instances per frame. The truck count (second panel) is lower and more variable, reflecting the intermittent presence of large vehicles in the scene.

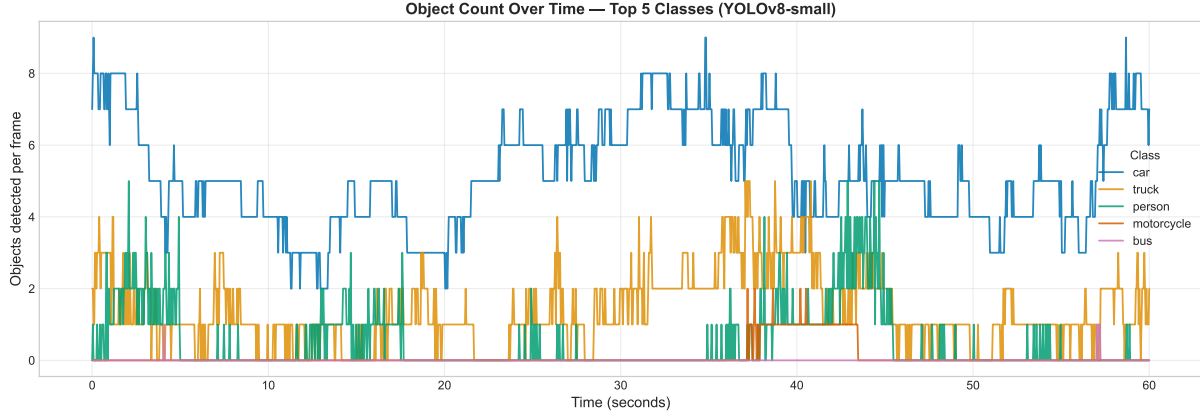


Figure 3.4: Per-frame object count time-series for the top-5 detected classes (car, truck, person, motorcycle, bus) over 1,501 frames (60 seconds). The car class is present in every frame; motorcycles appear only in a specific time window ($\approx$ frames 700–850). Generated by *Notebook 05*.

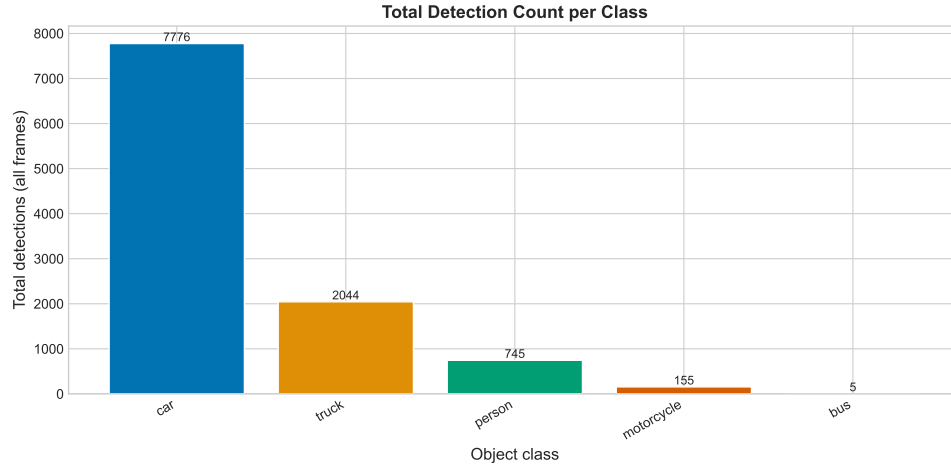


Figure 3.5: Total detection appearances per class across all 1,501 frames. Cars dominate with 7,776 total appearances. The y-axis is on a linear scale; the difference between cars (7,776) and buses (5) spans three orders of magnitude. Generated by *Notebook 05*.

3.4 Model Benchmarking

Violin Plot Interpretation. Figure 3.6 shows the latency distribution for all three models as violin plots. The violin plot encodes more information than a box plot: the width of the violin at any latency value indicates the density of observations at that value.

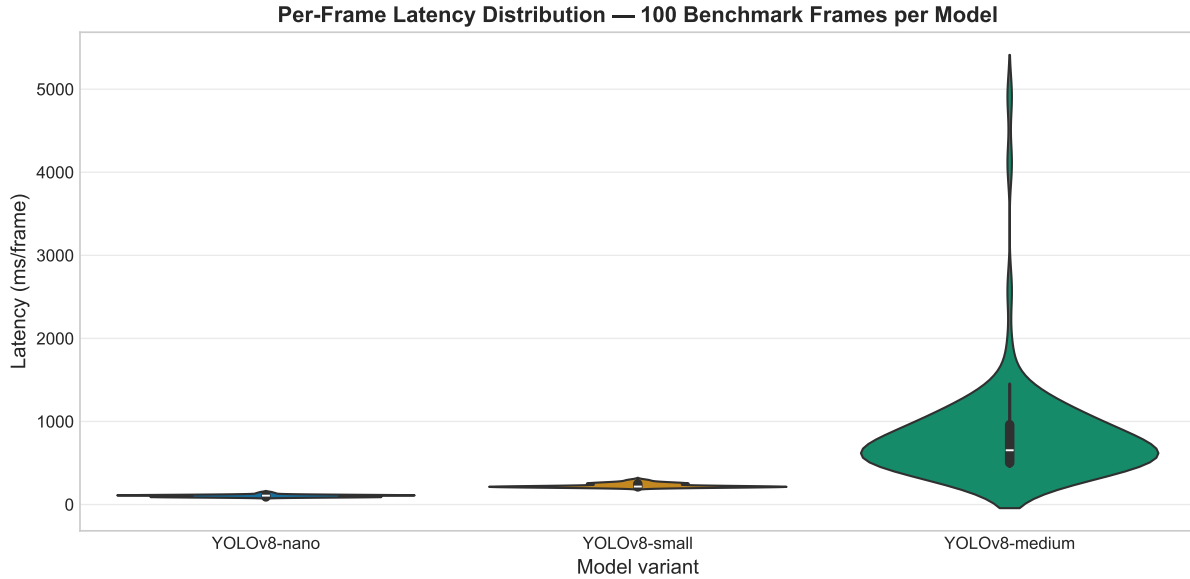


Figure 3.6: Latency distributions for YOLOv8-nano, small, and medium over 100 benchmark frames (640×480 px, CPU). Violin plots show the full distribution; overlaid box plots mark the median, IQR, and 1.5×IQR whiskers. Nano’s distribution is tight and near-symmetric. Small is mildly right-skewed. Medium has a dramatically heavy right tail ($p_{99} = 4,119$ ms), indicating frequent OS-induced stalls. Generated by *Notebook 06*.

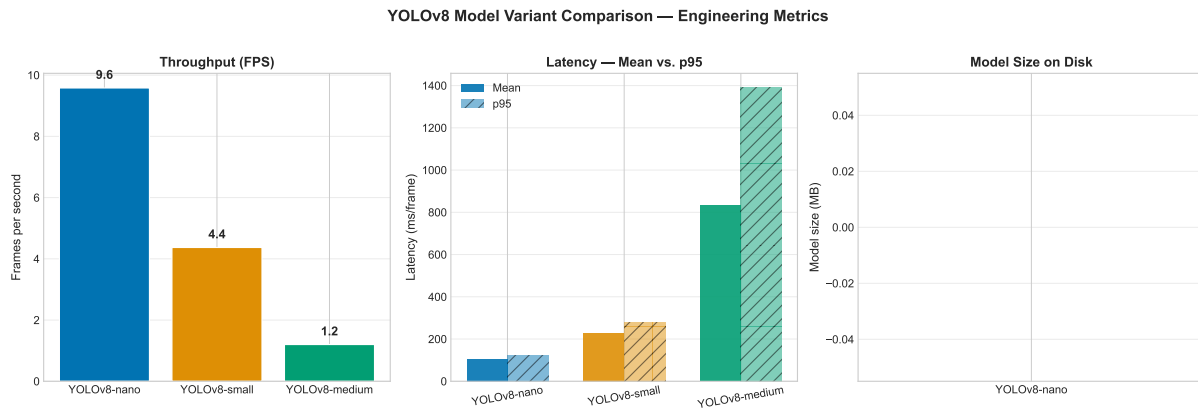


Figure 3.7: Three-panel benchmark comparison for all three YOLOv8 variants. Left: mean FPS (higher is better). Centre: mean latency with p95 error bar (lower is better). Right: on-disk model size in MB (smaller enables edge deployment). Generated by *Notebook 06*.

3.5 Model Size Comparison and Deployment Recommendations

Notebook 07 synthesises the benchmark results with a detection sensitivity proxy to produce a final model recommendation. It introduces the speed–size scatter plot (Figure 3.8), which plots mean FPS (y-axis) against parameter count (x-axis) for all three models, with point size encoding model file size.

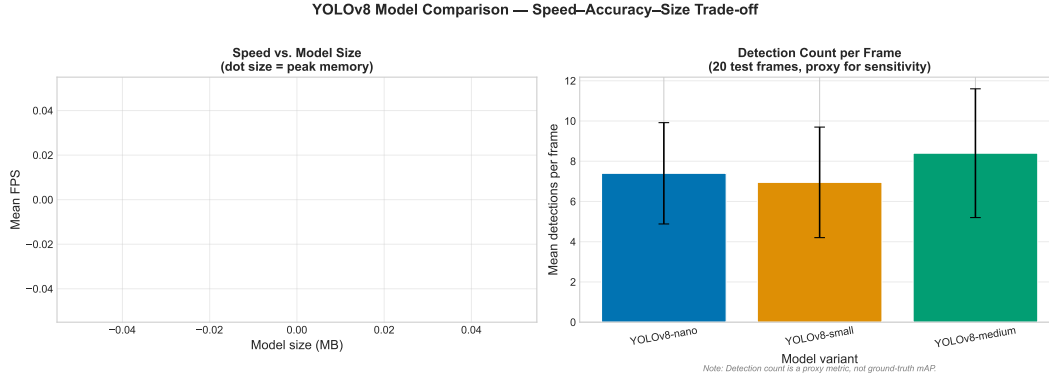


Figure 3.8: Speed–size trade-off scatter plot. Each point represents one YOLOv8 variant. The x-axis shows parameter count; the y-axis shows mean inference FPS (CPU). Point size is proportional to on-disk model file size. A “good” model lies in the upper-left (fast, small). Nano is the clear winner on speed; medium achieves higher detection sensitivity but at a $\sim 8\times$ FPS cost. Generated by *Notebook 07*.

The detection count proxy (mean detections per frame on 20 test frames) is shown alongside the benchmark metrics:

Model	Mean Det/Frame	Std Det/Frame
YOLOv8-nano	7.4	2.5
YOLOv8-small	7.0	2.7
YOLOv8-medium	8.4	3.2

The medium model detects 8.4 objects per frame vs. nano’s 7.4—a 13.5% higher count that is consistent with a richer feature representation. However, the wide standard deviation (3.2 vs. 2.5) and the explicit caveat that this proxy does not distinguish true positives from false positives mean this advantage should be interpreted cautiously.

References

- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*. Retrieved from <https://arxiv.org/abs/2004.10934>
- Buttazzo, G. C. (2011). *Hard real-time computing systems: Predictable scheduling algorithms and applications* (3rd ed.). New York, NY: Springer.
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition (cvpr)* (pp. 886–893). doi: 10.1109/CVPR.2005.177
- Girshick, R. (2015). Fast R-CNN. In *Proceedings of the IEEE international conference on computer vision (iccv)* (pp. 1440–1448). Retrieved from <https://arxiv.org/abs/1504.08083> doi: 10.1109/ICCV.2015.169
- Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition (cvpr)* (pp. 580–587). Retrieved from <https://arxiv.org/abs/1311.2524> doi: 10.1109/CVPR.2014.81
- IHS Markit. (2022). *Global surveillance camera market report*. (Market research report. Cited in media coverage at <https://www.securityinformed.com/news/1-billion-surveillance-cameras-2021.html>)
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. (Version 8.0.0. MIT Licence. Model documentation at <https://docs.ultralytics.com/>)
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems (neurips)* (Vol. 25, pp. 1097–1105). Retrieved from https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In *Proceedings of the European conference on computer vision (eccv)* (pp. 740–755). Retrieved from <https://arxiv.org/abs/1405.0312> doi: 10.1007/978-3-319-10602-1_48
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. In *Proceedings of the European conference on computer vision (eccv)* (pp. 21–37). Retrieved from <https://arxiv.org/abs/1512.02325> doi: 10.1007/978-3-319-46448-0_2
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern*

- recognition (cvpr)* (pp. 779–788). Retrieved from <https://arxiv.org/abs/1506.02640> doi: 10.1109/CVPR.2016.91
- Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. In *Proceedings of the ieee conference on computer vision and pattern recognition (cvpr)* (pp. 7263–7271). Retrieved from <https://arxiv.org/abs/1612.08242> doi: 10.1109/CVPR.2017.690
- Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*. Retrieved from <https://arxiv.org/abs/1804.02767>
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems (neurips)* (Vol. 28, pp. 91–99). Retrieved from <https://arxiv.org/abs/1506.01497>
- Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. In *Proceedings of the ieee conference on computer vision and pattern recognition (cvpr)* (Vol. 1, pp. 511–518). doi: 10.1109/CVPR.2001.990517